

Pesquisa Comparativa entre IDEs para Desenvolvimento de Firmware

Sumário

1	Contexto do Projeto	2
1.1	Sensores e atuadores	2
1.2	Sistema de energia	2
1.3	Requisitos que a IDE deve suportar	3
2	IDEs Analisadas	3
2.1	Arduino IDE	3
2.2	PlatformIO (extensão do VSCode)	3
3	Tabela Comparativa	4
4	IDE Escolhida	4
5	Justificativa	4
5.1	Arquitetura dual-core no projeto	4
5.2	MCPWM é necessário para controle preciso dos motores	4
5.3	WebSocket nativo para telemetria em tempo real	5
5.4	Integração direta com GitHub	5
5.5	Debug integrado	5
6	Guia de Instalação e Execução	5
6.1	Pré-requisitos	5
6.2	Passo 1 — Instalar o Visual Studio Code	5
6.3	Passo 2 — Instalar a extensão PlatformIO	5
6.4	Passo 3 — Criar um novo projeto para o ESP32-S3	6
6.5	Passo 4 — Configurar o platformio.ini	6
6.6	Passo 5 — Sugestão de estrutura de pastas do projeto	6
6.7	Passo 6 — Compilar e gravar o firmware	7
6.8	Passo 7 — Instalar bibliotecas necessárias	7
7	Glossário de Siglas	7
8	Referências	9

1 Contexto do Projeto

O microcontrolador escolhido pela equipe é o **ESP32-S3-WROOM-1-N16R8**, com as seguintes características relevantes para o firmware:

Tabela 1: Especificações do ESP32-S3-WROOM-1-N16R8

Característica	Valor
Processador	Xtensa dual-core LX7, até 240 MHz
SRAM interna	512 KB
Flash externa	16 MB (Quad SPI)
PSRAM externa	8 MB (Octal SPI, in-package)
Sistema operacional	FreeRTOS (nativo no ESP-IDF)
Conectividade	Wi-Fi 802.11b/g/n, Bluetooth 5
Periféricos	MCPWM, I2C, SPI, UART, ADC, USB

1.1 Sensores e atuadores

Tabela 2: Componentes do Micromouse e suas interfaces

Componente	Função	Interface
VL53L0X	Sensor de distância ToF (detecção de paredes)	I2C
MPU-6000	IMU — giroscópio + acelerômetro (odometria)	SPI ou I2C
L298N	Driver de motor H-Bridge (controle dos motores)	GPIO + PWM
BMS-20A-3S-S	Proteção do pacote de bateria 3S (12.6V / 20A)	— (hardware)

1.2 Sistema de energia

O sistema utiliza um pacote de 3 células de lítio em série (3S) gerenciado pelo BMS-20A-3S-S, com as seguintes características relevantes para o firmware:

Tabela 3: Parâmetros do BMS-20A-3S-S

Parâmetro	Valor
Tensão totalmente carregado	12.6V
Tensão nominal	11.1V
Tensão mínima por célula	2.5V
Corrente contínua máxima	20A
Proteção de sobrecorrente	60A (disparo)
Temperatura de operação	-40°C a 85°C

O firmware precisa ler a tensão do pacote via **ADC** do ESP32-S3 (através de um divisor de tensão no hardware) e converter para porcentagem de carga.

Atenção ao hardware: o BMS fornece 12.6V, mas o ESP32-S3 opera em 3.3V. O hardware deve incluir um regulador de tensão 12.6V → 3.3V para alimentar o microcontrolador. O ADC do ESP32-S3 também opera em no máximo 3.3V, portanto o divisor de tensão para leitura da bateria será necessário.

1.3 Requisitos que a IDE deve suportar

- FreeRTOS com tasks fixadas em cores específicos (*pinned tasks*)
- Arquitetura dual-core (Core 0 — tempo real / Core 1 — navegação e telemetria)
- Controle de motores via MCPWM (Motor Control PWM)
- Comunicação WebSocket sobre Wi-Fi para telemetria em tempo real
- Leitura de sensores via I2C (VL53L0X) e SPI ou I2C (MPU-6000)
- Leitura de tensão de bateria via ADC (pacote 3S — BMS-20A-3S-S)
- Integração nativa com GitHub
- Suporte ao ESP-IDF (framework oficial da Espressif)

2 IDEs Analisadas

2.1 Arduino IDE

Suporta o ESP32 através do pacote `arduino-esp32` instalado via Boards Manager.

O suporte ao ESP32 funciona por meio de uma camada de abstração (HAL) que traduz funções do Arduino (`digitalWrite`, `analogWrite`) para chamadas do ESP-IDF internamente.

Documentação oficial: <https://docs.arduino.cc>

2.2 PlatformIO (extensão do VSCode)

Ecossistema de desenvolvimento para sistemas embarcados que funciona como extensão do Visual Studio Code. Suporta mais de 1.500 placas e 40 plataformas diferentes.

Quando configurado com o framework `espidf`, o PlatformIO compila o projeto diretamente com o ESP-IDF da Espressif, sem camadas de abstração, dando acesso total ao FreeRTOS, drivers de periféricos, pilha de rede Wi-Fi e todos os recursos do ESP32-S3.

Documentação oficial: <https://docs.platformio.org>

ESP-IDF: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/>

3 Tabela Comparativa

Tabela 4: Comparativo entre Arduino IDE e PlatformIO + ESP-IDF

Critério	Arduino IDE	PlatformIO + ESP-IDF
FreeRTOS com pinned tasks	Parcial	Completo
Dual-core (Core 0 e Core 1)	Sem garantias	Nativo
MCPWM (controle de motor)	Não exposto	API nativa
WebSocket nativo ESP-IDF	Não disponível	Biblioteca oficial
I2C com múltiplos dispositivos	Disponível	Disponível
ADC para leitura de bateria	Disponível	Disponível
Suporte ao ESP32-S3	Limitado	Completo
Acesso direto ao ESP-IDF	Bloqueado pela HAL	Direto
Debug avançado	Básico	Integrado ao VSCode
Gerenciamento de PSRAM	Difícil	Configurável
Integração com GitHub	Ausente	Nativa via VSCode
Gerenciamento de bibliotecas	Simples	Avançado
Curva de aprendizado	Baixa	Média
Documentação da comunidade	Extensa	Extensa
Tutoriais para iniciantes	Muitos	Muitos

4 IDE Escolhida

PlatformIO (extensão do VSCode) com framework ESP-IDF.

5 Justificativa

A escolha do PlatformIO com ESP-IDF é justificada pelos seguintes pontos:

5.1 Arquitetura dual-core no projeto

O firmware do Micromouse divide o processamento entre dois núcleos:

- **Core 0** — leitura de sensores, controle PID dos motores e odometria (tempo real)
- **Core 1** — algoritmos da área de software, atualização do mapa e envio de telemetria via WebSocket

Essa separação exige o uso de `xTaskCreatePinnedToCore`, função nativa do FreeRTOS que não é adequadamente suportada pelo Arduino IDE no ESP32-S3.

5.2 MCPWM é necessário para controle preciso dos motores

O L298N é controlado via sinais PWM de alta resolução. O ESP-IDF oferece a API MCPWM (Motor Control PWM), projetada especificamente para controle de motores, com controle de frequência, duty cycle e fase. Essa API não é exposta pela camada de abstração do Arduino IDE.

5.3 WebSocket nativo para telemetria em tempo real

O projeto exige envio de dados de telemetria em tempo real para um sistema web. O ESP-IDF oferece a biblioteca `esp_websocket_client` oficial da Espressif. No Arduino IDE, seria necessário depender de bibliotecas de terceiros sem suporte oficial para o ESP32-S3.

5.4 Integração direta com GitHub

O PlatformIO roda dentro do VSCode, que possui integração nativa com Git e GitHub. Isso facilita o gerenciamento de issues, branches e pull requests.

5.5 Debug integrado

O PlatformIO oferece debug via JTAG integrado ao VSCode, permitindo inspecionar variáveis, pausar a execução e monitorar as tasks do FreeRTOS em tempo real. O Arduino IDE possui apenas debug por Serial, insuficiente para um sistema com múltiplas tasks paralelas.

6 Guia de Instalação e Execução

6.1 Pré-requisitos

- Sistema operacional: Windows 10/11, macOS ou Linux

6.2 Passo 1 — Instalar o Visual Studio Code

1. Acesse <https://code.visualstudio.com>
2. Baixe o instalador para o seu sistema operacional
3. Execute o instalador e siga as instruções padrão

6.3 Passo 2 — Instalar a extensão PlatformIO

1. Abra o VSCode
2. Clique no ícone de extensões na barra lateral (ou **Ctrl+Shift+X**)
3. Pesquise por **PlatformIO IDE**
4. Clique em **Install**
5. Aguarde a instalação completa (pode levar alguns minutos)
6. Reinicie o VSCode quando solicitado

6.4 Passo 3 — Criar um novo projeto para o ESP32-S3

1. Clique no ícone do PlatformIO na barra lateral
2. Clique em **New Project**
3. Preencha:
 - **Name:** micromouse
 - **Board:** Espressif ESP32-S3-DevKitC-1
 - **Framework:** Espidf
4. Clique em **Finish** e aguarde o download do ESP-IDF (~500 MB na primeira vez)

6.5 Passo 4 — Configurar o platformio.ini

Substitua o conteúdo do arquivo `platformio.ini` na raiz do projeto por:

```
1 [env:esp32-s3-devkitc-1]
2 platform = espressif32
3 board = esp32-s3-devkitc-1
4 framework = espidf
5
6 ; Configuracao de memoria
7 board_build.flash_size = 16MB
8 board_build.psram_type = opi
9
10 ; Monitor serial
11 monitor_speed = 115200
12
13 ; Porta serial (ajustar conforme o sistema)
14 ; Windows: upload_port = COM3
15 ; Linux/Mac: upload_port = /dev/ttyUSB0
```

Listing 1: `platformio.ini` para o Micromouse

Os valores de configuração de memória foram definidos com base no datasheet do módulo ESP32-S3-WROOM-1 v1.8, considerando o part number N16R8 (16 MB Flash Quad SPI e 8 MB PSRAM Octal SPI).

6.6 Passo 5 — Sugestão de estrutura de pastas do projeto

```
1 micromouse/
2 |-- platformio.ini      <- configuracao da plataforma
3 |-- CMakeLists.txt     <- gerado automaticamente
4 |-- src/
5 |   |-- main.c          <- ponto de entrada
6 |   |-- contrato.h      <- structs entre Core 0 e Core 1
7 |   |-- core0/
8 |   |   |-- firmware.c  <- sensores, motores, PID
9 |   |-- core1/
10 |      |-- software.c   <- Flood Fill, mapa, WebSocket
```

```

11 | -- components/          <- bibliotecas customizadas
12 | -- managed_components/  <- bibliotecas do ESP-IDF

```

Listing 2: Estrutura de pastas sugerida

6.7 Passo 6 — Compilar e gravar o firmware

1. Conecte o ESP32-S3 via cabo USB
2. No VSCode, pressione **Ctrl+Alt+U** para compilar e gravar
3. Ou clique no ícone na barra inferior do PlatformIO
4. Para abrir o monitor serial: **Ctrl+Alt+S**

6.8 Passo 7 — Instalar bibliotecas necessárias

As bibliotecas para os sensores podem ser instaladas via ESP-IDF Component Manager. Adicione ao arquivo `idf_component.yml`:

```

1 dependencies:
2   idf: ">=5.0.0"
3   espressif/esp_websocket_client: ">=1.0.0"

```

Listing 3: `idf_component.yml`

Para o VL53L0X e MPU-6000, utilize os drivers disponíveis no repositório oficial da Espressif ou implemente via I2C/SPI diretamente com a API do ESP-IDF.

7 Glossário de Siglas

Sigla	Significado	Explicação simples
ADC	Analog to Digital Converter	Conversor que lê a tensão da bateria e transforma em número que o código entende
BMS	Battery Management System	Placa que protege a bateria contra sobrecarga, subdescarga e curto-circuito
ESP-IDF	Espressif IoT Development Framework	Framework oficial para programar o ESP32 em C, com acesso total ao hardware
FreeRTOS	Free Real-Time Operating System	Sistema operacional que permite rodar várias tarefas ao mesmo tempo no ESP32
GPIO	General Purpose Input/Output	Pinos do microcontrolador usados para enviar ou receber sinais digitais

Sigla	Significado	Explicação simples
HAL	Hardware Abstraction Layer	Camada do Arduino IDE que simplifica o hardware mas esconde funcionalidades avançadas
H-Bridge	Ponte H	Circuito do L298N que permite controlar a direção e velocidade dos motores
I2C	Inter-Integrated Circuit	Protocolo de comunicação serial com 2 fios, usado pelo VL53L0X e MPU-6000
IMU	Inertial Measurement Unit	Sensor (MPU-6000) que mede aceleração e rotação para saber a posição do mouse
JTAG	Joint Test Action Group	Interface de debug que permite pausar o programa e inspecionar variáveis em tempo real
LX7	Xtensa LX7	Nome da arquitetura do processador dual-core do ESP32-S3
MCPWM	Motor Control PWM	Periférico do ESP32 projetado para controle preciso de motores via sinais PWM
PID	Proportional Integral Derivative	Algoritmo de controle que corrige a trajetória e velocidade do mouse automaticamente
PSRAM	Pseudo-Static RAM	Memória RAM externa ao chip usada para dados maiores como buffers de telemetria
PWM	Pulse Width Modulation	Sinal que controla a velocidade dos motores variando o tempo ligado/desligado
SMP	Symmetric Multiprocessing	Uso simultâneo dos dois núcleos do processador para tarefas paralelas
SPI	Serial Peripheral Interface	Protocolo de comunicação serial mais rápido que o I2C, suportado pelo MPU-6000
SRAM	Static Random Access Memory	Memória rápida interna ao chip usada durante a execução do programa
3S	3 células em Série	Configuração de bateria com 3 células de lítio em série resultando em 12.6V
ToF	Time of Flight	Tecnologia do VL53L0X que mede distância pelo tempo que a luz leva para ir e voltar
UART	Universal Asynchronous Receiver Transmitter	Protocolo serial simples usado para comunicação com o monitor serial durante o desenvolvimento

Sigla	Significado	Explicação simples
WebSocket	—	Protocolo que mantém uma conexão aberta e contínua entre o mouse e o servidor web

8 Referências

- **Arduino IDE — Documentação oficial**
<https://docs.arduino.cc/software/ide-v2>
- **arduino-esp32 — Pacote ESP32 para Arduino**
<https://github.com/espressif/arduino-esp32>
- **PlatformIO — Documentação oficial**
<https://docs.platformio.org/en/latest/>
- **ESP-IDF — Guia de programação para ESP32-S3**
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/>
- **FreeRTOS SMP no ESP32-S3 (dual-core)**
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-guides/freertos-smp.html>
- **ESP-IDF MCPWM — Controle de motores**
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/mcpwm.html>
- **ESP WebSocket Client — Biblioteca oficial**
<https://github.com/espressif/esp-idf/tree/master/examples/protocols/websocket>
- **Datasheet ESP32-S3-WROOM-1 v1.8**
https://www.espressif.com/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf
- **Datasheet BMS-20A-3S-S — Placa de proteção de bateria 3S**
Shenzhen Global Technology Co., Ltd. — disponível na pasta hw/datasheets/ do repositório